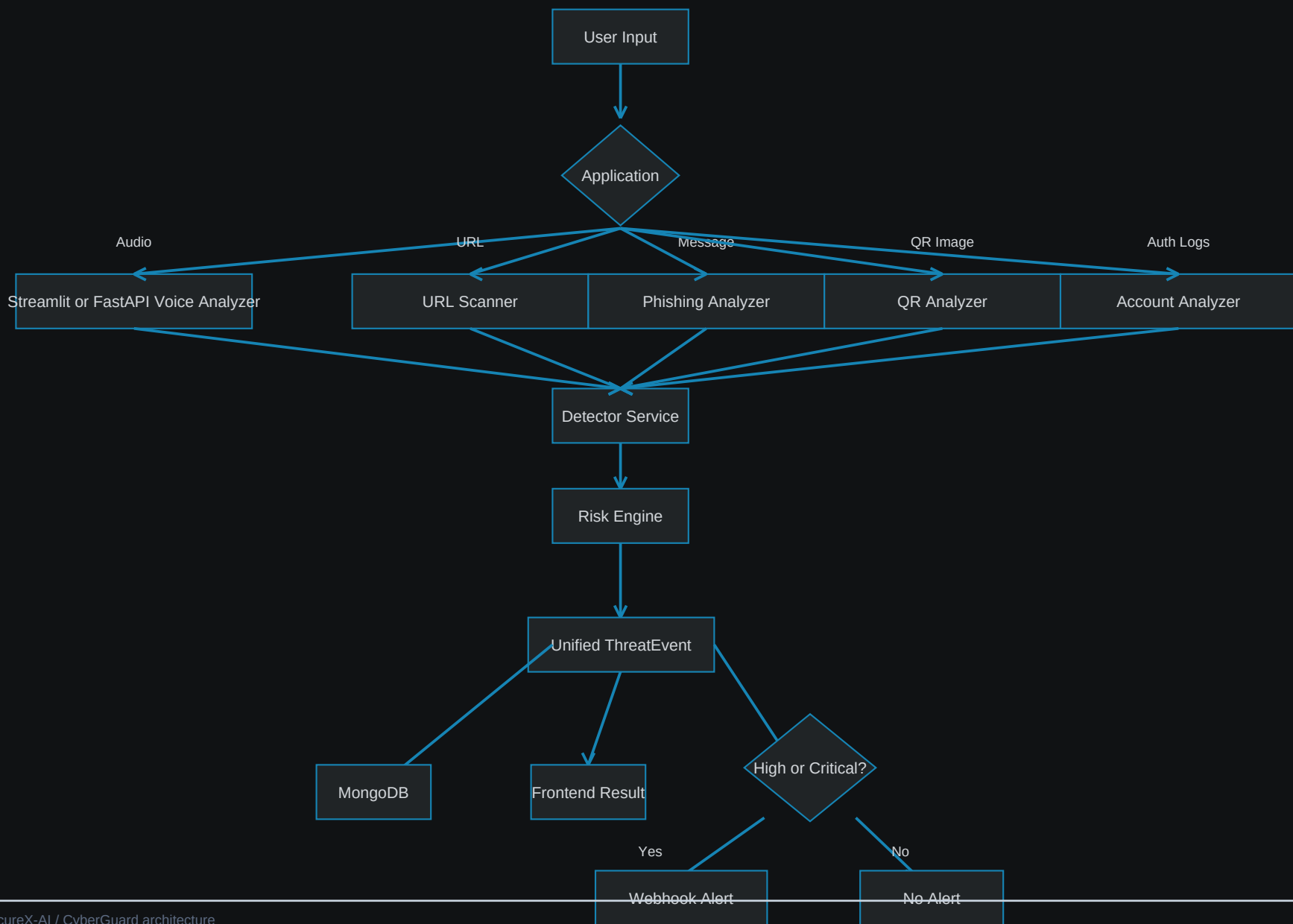


SECUREX-AI

Project Architecture

Graphical structure, feature workflows, and deployment map

Voice authenticity, phishing, URL, QR, and account-threat detection



Complete project structure

The repository contains a standalone voice product and a broader CyberGuard platform. The root src package is the canonical voice engine used by both the Streamlit app and the backend voice service.

```
SecureX-ai/ |-- app.py Streamlit VoxGuard voice app |-- streamlit_app.py Streamlit CyberGuard dashboard |-- train.py voice model training CLI |--
model.joblib serialized classifier |-- requirements.txt Python dependencies |-- docker-compose.yml MongoDB + backend + frontend |--
Dockerfile.backend FastAPI image |-- Dockerfile.frontend React/NGINX image |-- README.md setup and usage documentation |--
SNOWFLAKE_DEPLOYMENT_GUIDE.md Snowflake deployment notes |-- snowflake_setup.sql Snowflake objects/setup |-- data/sample_audio/ labeled
demo/training audio |-- scripts/generate_samples.py synthetic sample generator |-- src/ | |-- audio_preprocessing.py denoise and clean audio | |--
feature_extraction.py pitch, spectral, MFCC, chroma features | |-- inference.py model inference and probabilities | |-- model.py scaler + Random
Forest training | |-- transcription.py Whisper speech-to-text | `-- scam_analyzer.py transcript scam rules |-- tests/test_pipeline.py voice
pipeline tests `-- cyberguard/ |-- backend/ | |-- main.py FastAPI entry point | |-- config.py environment configuration | |-- risk_engine.py
shared score-to-level mapping | |-- api/ auth, voice, URL, QR, phishing, | | account, events, explain, health | |-- services/ detector and
integration logic | |-- models/ unified ThreatEvent schema | |-- db/ Motor/MongoDB repository | |-- middleware/ API key and rate limiting | `--
tests/ backend unit tests |-- frontend/ | |-- src/App.jsx tabs and refresh lifecycle | |-- src/services/api.js Axios API client | `--
src/components/ dashboard feature panels |-- dashboard/app.py additional dashboard entry point `-- src/ alternate/legacy package location
```

Feature workflows

All detector paths converge on the same result pattern: validate input, collect evidence, calculate risk, persist the event, and optionally alert.

Feature	Input	Processing	Output
Voice authenticity	Audio file or microphone recording	Denoise -> acoustic features -> Random Forest -> Whisper -> scam rules	AI/Human verdict, confidence, transcript, acoustic metrics, overall risk
URL reputation	HTTP/HTTPS URL	Block-list + Safe Browsing + VirusTotal + public-page inspection	Evidence, risk score, recommendations
QR phishing	QR image	OpenCV decode -> URL analysis -> redirect and impersonation checks	Decoded URL, QR evidence, risk level
Phishing text	Email, SMS, or chat text	Keyword, formatting, sender-context, and email-header heuristics	Phishing score, matched indicators, guidance
Account takeover	Authentication log lines	Failed-login counts + AbuseIPDB + geolocation + impossible travel	Account risk, IP/geographic evidence, recommendations

Runtime and deployment flow

Use the standalone Streamlit workflow for voice review, or run the full platform when analysis results must be shared, stored, queried, exported, and alerted on.

Local development	Production-style deployment
<pre>streamlit run app.py Optional backend: uvicorn cyberguard.backend.main:app --reload Optional frontend: cd cyberguard/frontend npm run dev</pre>	<pre>docker compose up -d --build React frontend: :3000 FastAPI backend: :8000/api MongoDB: :27017</pre>

Event lifecycle

A detector creates a ThreatEvent. The API saves it in MongoDB, returns it to the client, and schedules a webhook for HIGH or CRITICAL events. The React dashboard refreshes event history every 30 seconds. A retention task removes expired records according to RETENTION_DAYS.

Configuration surfaces

MONGODB_URI, SAFE_BROWSING_API_KEY, VIRUSTOTAL_API_KEY, ABUSEIPDB_API_KEY, CYBERGUARD_API_KEY, JWT_SECRET, ALERT_WEBHOOK_URL, ALERT_MIN_RISK_LEVEL, RETENTION_DAYS, and rate-limit settings are read from environment variables.